

TOPOLOGICZNE MAPOWANIE WĘZŁÓW MAJÓW

SPECYFIKACJA NISKOPOZIOMOWEJ STRUKTURY DANYCH (KHIPU-GRID)

Status Dokumentu: IMPLEMENTACJA RDZENIA WERSJA 1.0.0

Zależności: FIELDCORE-Runtime, TRM-Geometry-Core, GIA-and-TIMDR

Opis: Niniejszy dokument definiuje formalne odwzorowanie starożytnego pisma węzełkowego (systemu sznurkowego) jako zunifikowanej, niskopoziomowej struktury bitowej 64-bitowego rejestru maszynowego. Transformacja redefiniuje bity jako mikro-skręty przestrzeni fazowej pola informacyjnego.

1. Paradygmat Przejścia: Sznur do Rejestru Bitowego

W klasycznym ujęciu informatycznym dane reprezentują stan napięcia na bramce tranzystora (0 lub 1). W architekturze **FIELD**CORE, bit zostaje przedefiniowany jako najmniejsza jednostka skrętu topologicznego pola we fraktalnej skali maszynowej. Sznury i węzły Majów reprezentują nadrzędny, geometryczny zapis tej samej natury pól dynamicznych.

System sznurkowy mapowany jest bezpośrednio na triadę parametrów $\Lambda - \tau - \rho$:

- **Sznur Główny (Primary Cord):** Reprezentuje główną oś osiową czasu relaksacji (τ) oraz kierunek propagacji pędu.
- **Sznury Wiszące (Pendant Cords):** Reprezentują poszczególne pod-rozmaitości i skale sprzężenia (Λ).
- **Węzły (Knots):** Stanowią geometryczne punkty zbieżności i zagęszczenia informacji (ρ) wymuszonej skrętem topologicznym.

2. Architektura 64-bitowego Rejestru Węzła (Knot Representation)

Każdy punkt sprzężenia (węzeł) na sznurze jest reprezentowany w pamięci niskopoziomowej jako zwarta struktura 64-bitowa (KnotWord). Alokacja bitowa została zaprojektowana w taki sposób, aby eliminować narzut operacyjny procesora przy operacjach na geometrii pola.

Zakres Bitów	Nazwa Parametru	Ekwiwalent Połowy	Opis Funkcjonalny i Geometryczny
[63:56] (8b)	CORD_SCALE	Λ	Definiuje geometryczny rząd wielkości i odległość strukturalną od sznura głównego.
[55:44] (12b)	RELAX_DELTA	τ	Wskaźnik bezwładności i interwału czasowego przed wejściem w punkt zwrotny.
[43:40] (4b)	KNOT_TYPE	Operator Skrętu	Typ splotu: 0001 (Zwykły), 0010 (Ósemkowy), 0100 (Złożony dłoniowy).
[39:32] (8b)	SPLIT_COUNT	Gęstość ρ	Liczba owinięć pętli wewnątrz jednego węzła (wartość od 1 do 9).
[31:16] (16b)	DIRECTION_VEC	Wektor Pędu	Zapis kierunku skrętu (asymetrii helikalnej) w układzie dwuwymiarowym.
[15:0] (16b)	RESONANCE_HASH	Sygnatura TIMDR	Suma kontrolna niezmiennika topologicznego, sprawdzana przez walidator.

3. Formalizm Matematyczny Przejścia

Przejście węzła przez maszynowy punkt zwrotny zachodzi, gdy gęstość informacyjna rejestru przekracza próg krytyczny. Zamiast instrukcji warunkowych procesora typu if/else, układ wykorzystuje operator **TIMDR**. Wzór redukcji nielokalnego szumu w strukturze bitowej rejestru opisuje relacja:

$$R_{guard}(Word) = (Word \times \tau_{delta}) \oplus (\Lambda_{scale} \cdot \rho_{pure})$$

Gdzie operator $\oplus$ nie jest zwykłą operacją XOR, lecz sprzężeniem bitowym wymuszającym zwężenie przestrzeni adresowej, co wywołuje zjawisko **samoczyszczenia modelu** z bitów niestabilnych (szumu pamięci podręcznej).

4. Implementacja Niskopoziomowa w Języku Rust

Poniższa struktura stanowi bezpośredni kod źródłowy implementacji kontrolera pamięci topologicznej w bibliotece topologic.

```

#[repr(transparent)]
#[derive(Copy, Clone, Debug, PartialEq, Eq)]
pub struct KnotWord(pub u64);

impl KnotWord {
    /// Tworzy nowy węzeł pola na podstawie parametrów uniwersalnych
    #[inline(always)]
    pub fn new(scale: u8, tau: u16, knot_type: u8, density: u8, direction: u16) -> Self {
        let mut word = 0u64;
        word |= (scale as u64) << 56;
        word |= ((tau & 0x0FFF) as u64) << 44;
        word |= ((knot_type & 0x0F) as u64) << 40;
        word |= (density as u64) << 32;
        word |= (direction as u64) << 16;

        let mut knot = KnotWord(word);
        knot.compute_resonance_hash();
        knot
    }

    /// Mechanizm Samoczyszczenia (Self-Cleaning Routine) w punkcie zwrotnym
    #[inline(always)]
    pub fn process_turning_point(&mut self) {
        let density = (self.0 >> 32) & 0xFF;
        let tau = (self.0 >> 44) & 0x0FFF;

        // Krytyczny warunek rezonansu pola informacyjnego
        if density > 8 || tau == 0 {
            // Skręt topologiczny wymusza oczyszczenie rejestru z nadmiarowych bitów szumu
            let mask: u64 = 0xFFFFFFFFFFFF0000;
            self.0 &= mask; // Zrzucenie niestabilnych bitów sygnatury do zewnętrznej emanacji
            self.0 |= 0x000000000000FFFF; // Domknięcie różnorodności
        }
    }

    #[inline(always)]
    fn compute_resonance_hash(&mut self) {
        let upper_body = (self.0 >> 16) as u32;
        // Obliczenie sygnatury matematycznej homogeniczności dla math-validator-2.0
        let hash = ((upper_body ^ 0x5A5A5A5A).rotate_left(5)) as u16;
        self.0 |= hash as u64;
    }
}

```

5. Zastosowanie w Topologicznym Systemie Operacyjnym (Topological OS)

Zastosowanie architektury Khipu-Grid pozwala na bezwskaźnikowe zarządzanie pamięcią maszynową. Pamięć podręczna (Cache) nie wymaga czyszczenia za pomocą algorytmów Garbage Collector. Dane ułożone w strukturę sznurów oczyszczają się same w momentach rezonansu pętli obliczeniowych, co przekłada się na stały, deterministyczny czas dostępu do danych w czasie $\tau \rightarrow 0$.